Architectural Evolution of the Stela Skills Subsystem: From Mechanical Logging to Autonomous Forensic Intelligence

I. Executive Strategic Analysis

The Stela platform represents a paradigm shift in software operational governance, prioritizing "Governance as Product" over traditional ad-hoc development methodologies.[1] At its core, Stela operates on a stateless architecture where context—the operational awareness of the system—is not retained in memory but is strictly serialized into file-based contracts. This philosophy, encapsulated in the maxim "Context is not memory; it is a file" (docs/CONTEXT.md), necessitates rigorous mechanisms for capturing, verifying, and rehydrating knowledge.

The recent execution of **DP-OPS-0011** marked the inauguration of the **Skills Subsystem**, a critical infrastructure component designed to capture "crystallized intelligence"—reusable patterns of successful execution—and store them as S-LEARN artifacts. This transition was intended to move the platform from a purely reactive stance, where problems are solved *de novo* in every session, to an accumulative learning stance, where solutions are cached and re-deployed.

However, a comprehensive forensic analysis of the DP-OPS-0011-RESULTS [1] against the strategic mandate outlined in *Enhancing Stela Skills Subsystem* [1] reveals a significant functional delta. While DP-OPS-0011 successfully deployed the mechanical "plumbing" of the harvest workflow (the ability to create, draft, and promote skills), it failed to implement the "intelligent" operational logic required for high-fidelity autonomy. The current system remains a passive logging utility, reliant on operator manual entry and devoid of the forensic capability to analyze git state, infer context, or automatically categorize lessons.

This report serves as the definitive architectural blueprint and execution directive for **DP-OPS-0012**. Its objective is to close the gap between the current "dumb" harvest mechanism and the "Autonomous Harvest Engine" envisioned in the strategic roadmap. This upgrade will introduce git-based forensic analysis, semantic auto-classification, and rigorous storage hygiene to the platform. The report culminates in the formal Dispatch Packet (DP-OPS-0012), constructed strictly according to the TASK.md invariants, to execute these enhancements on the main branch at HEAD 827e7f32e.

II. Constitutional Framework and Operational

Invariants

To properly contextualize the architectural requirements of DP-OPS-0012, one must first deconstruct the constitutional framework of the Stela repository. The platform is not merely a collection of code; it is a governed system where "Truth" is defined by specific text files, and deviation from these files constitutes "System Drift."

2.1 The Philosophy of Stateless Intelligence and Context Rehydration

Stela's architecture is predicated on the elimination of "tribal knowledge." In traditional DevOps environments, the rationale behind a configuration change or the steps to resolve a complex dependency conflict often reside solely in the mind of the operator. Stela classifies this as a failure mode.[1]

The *Constitution* (TRUTH.md) and *Context Manifest* (docs/CONTEXT.md) establish a rigor where the "Context" is a finite, observable set of artifacts.[1]

- **The Invariant:** "If it is not in the Context, it does not exist."
- **The Consequence:** To introduce new knowledge (a skill) into the system, it must be serialized into a standard format (S-LEARN-XX) and registered in the immutable ledger.

The **Skills Subsystem** is the mechanism for this serialization. It transforms ephemeral "Work Log" data (which is time-bound and thread-specific) into durable "Library" data (which is timeless and universally applicable).

2.2 The Governance of the Harvest Engine

The "Harvest Engine" (ops/lib/skill/skill_lib.sh) functions as the write-head for this knowledge cache. Because it modifies the system's long-term memory, its operations are subject to strict governance rules defined in AGENTS.md and SKILL.md.[1]

2.2.1 The Context Hazard Rule

The most critical invariant governing the Skills Subsystem is the "Context Hazard".[1]

- **The Risk:** If every learned skill were added to the global ops/lib/manifests/CONTEXT.md, the context window would effectively become a "junk drawer" of low-signal information. An agent tasked with a simple CSS fix does not need to load the skill for "Database Sharding."
- **The Rule:** Skills must be "On-Demand Only." They are never loaded globally. They are invoked only when a Dispatch Packet (DP) explicitly requests them.
- **The Enforcement:** DP-OPS-0012 must harden the "Negative Context Check" in skill_lib.sh to programmatically prevent any S-LEARN-* ID from being registered in the global manifest.

2.2.2 The Provenance Requirement

Every artifact in Stela must have a traceable lineage. A skill cannot simply "appear"; it must be tied to the specific work session that generated it.[1]

- **Requirement:** The Harvest Engine must capture the Branch, Commit Hash, and Timestamp of the origination point.
- **Gap:** Currently, this relies on the operator correctly reporting the state. The "Autonomous" upgrade in DP-OPS-0012 must automate this via git commands to ensure irrefutable provenance.

III. Retrospective Forensic Analysis: DP-OPS-0011

The execution of DP-OPS-0011 ("Autonomous Skill Harvesting Engine") was a foundational success but an incomplete realization of the platform vision. By analyzing the *Results* artifact [1] and comparing it with the *Enhancing Stela Skills Subsystem* roadmap [1], we can identify specific architectural debts that DP-OPS-0012 must liquidate.

3.1 Gap Analysis: Mechanical vs. Intelligent Implementation

The primary deliverable of DP-OPS-0011 was the refactoring of ops/lib/skill/skill_lib.sh.[1] While the script was successfully expanded (adding 773 lines), it effectively digitized a manual process rather than automating intelligence.

Feature Domain	Mandate (Vision)	Executed State (DP-OPS-0011)	Variance / Gap
Lifecycle	Establish Harvest -> Draft -> Promote workflow.	**Shipped.** Workflow exists and functions.	None.
Git Forensics	Implement "Diff-Stat Heuristic" to auto-capture touched files.	**Missing.** User must manually input context or paste diffs.	**Critical.** Relies on fallible human memory.
Categorization	Auto-tag skills based on file paths (e.g., SECURITY.md -> security).	**Missing.** No semantic tagging logic exists.	**High.** Makes future retrieval difficult.

Evidence	Integrate ops/bin/dump to capture code snippets.	**Missing.** No connection between tools.	**Medium.** Reduces "Definition of Done" rigor.
Safety	Enforce "Negative Context" check.	**Partial.** Checks logic but needs hardening against "Shadow Skills."	**Low.** Functional but brittle.

3.2 The Storage Drift Incident

A critical finding in the DP-OPS-0011 execution log was the detection of **Storage Drift**.[1] The tools/verify_tree.sh script, which enforces the repository structure defined in TRUTH.md, reported warnings regarding unexpected directories in the local storage area:

- storage/archives
- storage/documentation
- storage/ToDo

Architectural Implication: According to TRUTH.md [1], the storage/ directory is reserved for "Temporary trash" and specific system paths (handoff, dumps). The presence of archives, documentation, and ToDo suggests that previous sessions have treated the local storage area as a persistent workspace. This violates the "Stateless" doctrine. If knowledge is important (Documentation), it belongs in docs/. If it is a task (ToDo), it belongs in TASK.md. If it is trash, it should be deleted.

Remediation Mandate for DP-OPS-0012:

The execution plan must include a "Hygiene Phase" to purge these unauthorized directories. This is not merely cleanup; it is the enforcement of the "Governance as Product" philosophy.

3.3 The "Primitive Utility" Critique

The vision document explicitly critiques the pre-DP-OPS-0011 state as a "primitive utility".[1] While DP-OPS-0011 advanced the tool, it did not solve the "Intelligence" problem. The current harvest command acts as a mad-libs generator—filling in templates with user strings. It does not *read* the repository.

To satisfy the requirement of an "Autonomous Harvest Engine," DP-OPS-0012 must invert this relationship. The tool must tell the operator what happened, rather than the operator telling the tool.

IV. Architectural Specification: The Intelligent Harvest Engine

Based on the gap analysis, the architectural requirement for DP-OPS-0012 is to enrich ops/lib/skill/skill_lib.sh with three specific functional modules: **The Forensic Module**, **The Semantic Tagging Module**, and **The Evidence Module**.

4.1 The Forensic Module (Git-Awareness)

The core hypothesis of the "Enhancing" vision is that *code changes reveal intent*.[1] If a session modifies ops/bin/open, the skill derived from that session is statistically likely to be about "Session Initialization" or "Tooling."

Technical Specification:

The harvest function must be updated to execute the following logic sequence:

1. **Branch Identification:** Determine the current working branch and the base branch (defaulting to main).
   Bash
   ```
   local branch=$(git rev-parse --abbrev-ref HEAD)
   local base="main"
   ```
2. **Diff-Stat Capture:** Execute git diff --stat $base...HEAD. This provides the quantitative "shape" of the work (e.g., "5 files changed, 20 insertions, 10 deletions").
3. **File List Capture:** Execute git diff --name-only $base...HEAD. This provides the raw data for semantic analysis.
4. **Injection:** These outputs must be injected into a new ## Provenance (Forensics) section in the generated draft.

Rationale:

This removes the cognitive load from the operator. They no longer need to manually list what they touched. The system provides irrefutable evidence of the "Surface Impact."

4.2 The Semantic Tagging Module (Auto-Classification)

Once the Forensic Module captures the file list, the system can apply deterministic rules to categorize the skill. This is not probabilistic AI "guessing"; it is logic-based classification.[1]

Logic Map (File Pattern -> Tag):

File Pattern	Derived Tag	Rationale

SECURITY.md or docs/security/*	security	Touches the security policy or manual.
ops/bin/* or ops/lib/*	platform-ops	Modifies the runtime machinery of Stela.
projects/*	project-payload	Modifies the actual deliverable code.
docs/* (excluding docs/ops)	documentation	Updates the manual without changing logic.
package.json or composer.json	dependency-mgmt	Changes the dependency graph.
.github/workflows/*	ci-cd	Modifies the gatekeeping or automation infrastructure.

Technical Specification:

The skill_lib.sh script must iterate through the captured file list, check against these patterns, and append unique tags to a ## Auto-Classification section in the draft.

4.3 The Evidence Module (Dump Integration)

The "Enhancing" document calls for "Evidence Collection via dump".[1] The ops/bin/dump tool is capable of serializing the repo state.

Technical Specification:

A new flag --capture-evidence should be added to the harvest command. When invoked:

1. It triggers ops/bin/dump with a narrowed scope (e.g., restricted to the files identified in the Forensic step, or a platform scope).
2. It saves this dump as a referenced artifact in storage/handoff/.
3. It adds a pointer to this dump in the Skill Draft.

Constraint Note: For DP-OPS-0012, we will prioritize the Forensics and Tagging modules. The Dump integration will be implemented as a "Reference Pointer" capability—allowing the user to attach a dump—rather than a full recursive dump of only touched files, to avoid excessive complexity in a bash script.

4.4 The "Heredoc Buffer" Pattern

The vision document identifies "Robust Input Handling" as a requirement.[1] The current script may struggle with multi-line inputs containing special characters.

Technical Specification:

DP-OPS-0012 will verify and enforce the use of read -d '' or similar heredoc patterns in the skill_lib.sh input collection loop to ensure that complex multi-line "Context" and "Solution" descriptions are captured faithfully without shell escaping errors.

V. Operational Hygiene and Risk Mitigation

Before the new code can be deployed, the platform must be stabilized. The "Storage Drift" identified in DP-OPS-0011 represents a "broken window" in the system's governance.

5.1 The Clean Sweep Strategy

To address the warnings from verify_tree.sh [1], DP-OPS-0012 must execute a ruthless cleanup of the storage/ directory.

Action Plan:

1. **Delete** storage/archives/: Historical logs should be effectively managed by git history or SoP.md. If "Museum" logs are required, they must be explicitly authorized in TRUTH.md or verify_tree.sh—but currently, they are flagged as drift, so they must go.
2. **Delete** storage/documentation/: All documentation belongs in docs/. Any documentation found in storage is, by definition, "Shadow Documentation" and must be purged or migrated.
3. **Delete** storage/ToDo/: The TASK.md file is the *Single Source of Truth* for active tasks. A separate "ToDo" directory fractures the task tracking.

5.2 Rollback and Safety

- **Risk:** The new harvest logic might break on edge cases (e.g., a fresh repo with no main branch, or a detached HEAD state).
- **Mitigation:** The script must include "Safety Checks" (e.g., verifying git is available and main exists) before attempting forensics. If forensics fail, it should degrade gracefully to manual entry rather than crashing.
- **Rollback:** The rollback plan for DP-OPS-0012 involves reverting ops/lib/skill/skill_lib.sh to the version committed in 827e7f32e and restoring the SKILL.md template.

VI. Dispatch Packet Construction (DP-OPS-0012)

The following section presents the **Dispatch Packet (DP-OPS-0012)**. This document is the executable instrument of the research findings. It is constructed in strict adherence to the TASK.md template [1] and targets the repository state at HEAD: 827e7f32e.

6.1 DP Formulation Strategy

- **Freshness Gate:** Locked to main at 827e7f32e.
- **Scope:** Strictly limited to ops/lib/skill/, SKILL.md, SoP.md, and the storage/ cleanup.
- **Execution Plan:** Divided into "State Hygiene" (cleanup) and "System Upgrade" (code changes).
- **Verification:** Leverages the standard suite (lint_truth, context_lint) plus the self-referential skill_lib.sh check.

DP-OPS-0012: Intelligent Harvest Engine & Storage Hygiene

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/dp-ops-0012-smart-harvest

Base HEAD: 827e7f32e

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/lib/skill/skill_lib.sh
  - SKILL.md
  - Enhancing Stela Skills Subsystem.md (if available in context)

II) SCOPE & SAFETY

- **Objective:** Upgrade the Skills Subsystem with autonomous git forensics (diff-stat, file list) and semantic tagging; remediate storage drift identified in DP-OPS-0011.
- **Non-Goals (optional, drift-killer):**
  - No changes to ops/bin/open or ops/bin/dump (consume only).
  - No changes to existing S-LEARN files (logic update only).

Target Files allowlist (hard gate)

- ops/lib/skill/skill_lib.sh
- SKILL.md
- SoP.md
- storage/ (cleanup only; no new permanent artifacts)

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.

- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** DP-OPS-0011 established the mechanical "Harvest/Promote" workflow but left the engine "dumb" (reliant on manual data entry).
- **Drift:** tools/verify_tree.sh reports warnings for storage/archives, storage/documentation, and storage/ToDo.
- **Desired State:** A "Smart" Harvest Engine that automatically captures git statistics and suggests tags, plus a clean storage/ directory.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **Storage Hygiene:** Delete storage/archives/, storage/documentation/, and storage/ToDo/ to resolve drift warnings.
2. **Upgrade skill_lib.sh (Forensics):**
   - Add a function to capture git diff --stat main...HEAD and git diff --name-only main...HEAD.
   - Inject this output into the "Provenance" section of the generated draft.
3. **Upgrade skill_lib.sh (Tagging):**
   - Implement semantic tagging logic based on touched files:
     - SECURITY.md or docs/security/* -> security
     - ops/* -> platform-ops
     - package.json -> dependency-management
     - projects/* -> project-payload
     - docs/* -> documentation
   - Inject tags into a new "## Auto-Classification" section in the draft.
4. **Upgrade SKILL.md:** Update the "Candidate Log" template to include the new "Tags" field.
5. **Update SoP:** Log the upgrade to the Skills Subsystem and the storage cleanup.

Required Content:

- The harvest command must still support manual overrides, but default to auto-discovery for provenance data.

C) CHANGELOG

- **Log (human-readable, 1–6 bullets):**
  - Purged unauthorized artifacts from storage/ to restore stateless hygiene.
  - Upgraded ops/lib/skill/skill_lib.sh with git diff forensics (auto-captures changed files/stats).
  - Added semantic auto-tagging to skill drafts based on file paths (security, ops, projects).
  - Updated SKILL.md templates to reflect new metadata fields.

D) PATCH / DIFF

- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- Context lint (use repo-canonical command):
  - Prefer: bash tools/context_lint.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when TRUTH/canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint_truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Default workflow: run ops/lib/skill/skill_lib.sh harvest to create a draft, review the candidate, then either promote with ops/lib/skill/skill_lib.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is permitted when explicitly requested by the DP or when harvest is not applicable. Use ops/lib/skill/skill_lib.sh "name" "context" "solution" and record the reason in RESULTS.
- If a DP requires skill capture, the DP Target Files allowlist must include SKILL.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
  - Promotion: command invoked plus grep -n "Promotion Packet:" SKILL.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.
  - Legacy capture: command invoked plus grep -n "Promotion Packet:" SKILL.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0012-RESULTS.md (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (when produced)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (when produced)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

No receipt/proof bundle = DISAPPROVE.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/.
- Results receipts land in storage/handoff/DP-OPS-0000-RESULTS.md (pattern).
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0012-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in

the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones
- [ ] Run repo-gates / required checks
- [ ] Commit/push/open PR/merge (operator-controlled)

3.1) THREAD TRANSITION (RESET / ARCHIVE RULE)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often "start new DP" or "await operator")
2. Reset sections **1) ACTIVE CONTEXT** and **2) CURRENT DISPATCH PACKET** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-

(If the Work Log gets too long, it's acceptable to keep only the most recent thread's entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

4) WORK LOG (TIMESTAMPED CONTINUITY)

Rule: This is not a transcript. It's the durable breadcrumb trail.

Expectation: Append a new timestamped entry on every DP closeout (no exceptions).

Each entry ends with: NEXT: <one single action>.

- *2026-02-01 16:30* — DP-OPS-0011: Added harvest/promote/check workflow to skill_lib, updated canon docs, and promoted S-LEARN-07. Verification: RUN (dump bundle; context_lint; lint_truth; lint_library; verify_tree WARN; dp_lint --test; skill_lib check).

Blockers: none. NEXT: operator review + commit.

- ** — DP-OPS-0012:. Verification:. Blockers: [none]. NEXT: [Action].

VII. Strategic Conclusion

DP-OPS-0012 represents a critical maturity inflection point for the Stela platform. By transitioning the Harvest Engine from a "dumb" logging tool to a "smart" forensic analyzer, the system significantly reduces the "Re-computation Tax" associated with skill documentation. The introduction of the Diff-Stat Heuristic and Semantic Tagging modules ensures that the skills captured are grounded in empirical reality—the code that actually changed—rather than fallible human memory. Furthermore, the rigorous enforcement of storage hygiene re-asserts the "Stateless" constitution, preventing the accrual of unmanaged debt in the local environment. This Dispatch Packet provides the complete instruction set to execute this transition with zero ambiguity and maximum traceability.

Works cited

1. dump-full-main-827e7f32e.txt